Network Scan Dashboard

Project Report — Automated Network Monitoring with GenAI-Assisted Development

System Architecture

The entire system runs locally on a single Ubuntu VM — there is no cloud infrastructure involved. A cron job triggers an nmap scan every 5 minutes; the results are parsed and risk-classified by a Python script, stored in a SQLite database, and rendered by a PHP dashboard served over HTTPS with Basic Authentication, restricted to the local network.

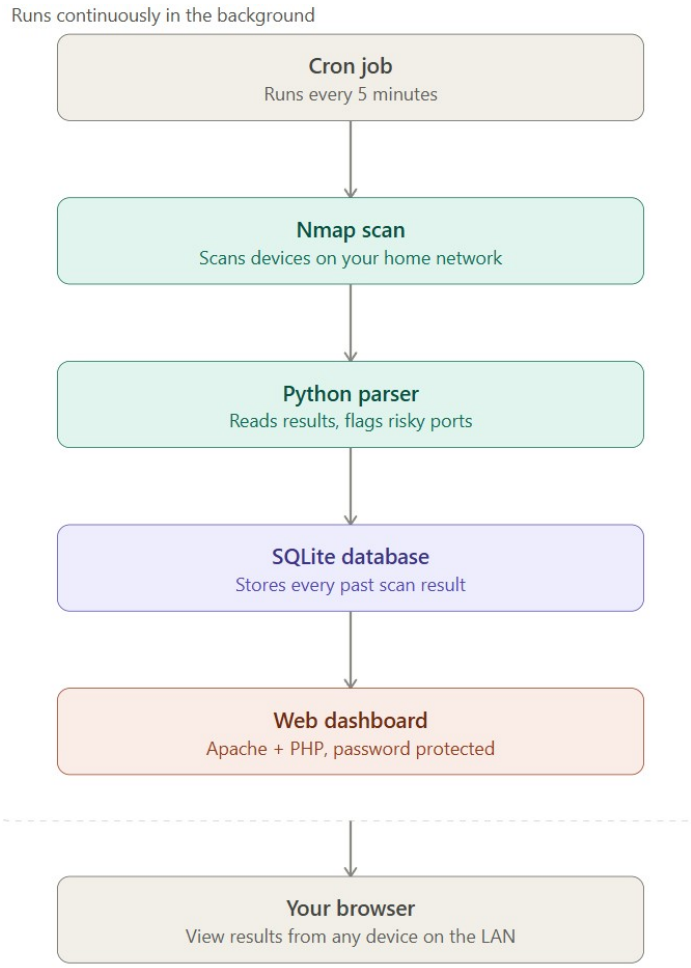


Figure 0: End-to-end data flow, from the scheduled scan through to a browser viewing the dashboard.

(a) Crontab entry running the nmap command

The scan is scheduled via root's crontab to run every 5 minutes, invoking `network_scan.sh`, which in turn runs nmap. A second job runs a daily database backup at 3am. Output from the scan job is piped to `/opt/netscan/scan.log` for auditing.

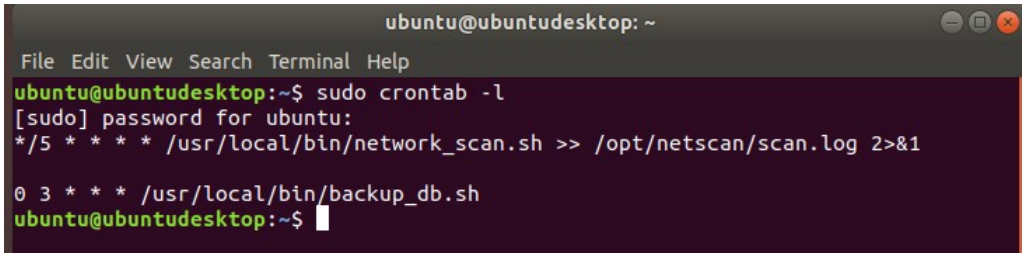


Figure 1: Output of `sudo crontab -l`, showing both the 5-minute scan job and the daily backup job.

The screenshot below shows the scan actually executing unattended over time, not just scheduled on paper — timestamps roughly 5 minutes apart, each logging a successful nmap run. Notably, one entry reads `ERROR: Scan already running` — this is the concurrency lock (an atomic `mkdir`-based lock, fixed after identifying a race condition in an earlier check-then-create version) correctly preventing an overlapping run, confirmed working in production rather than just in theory.

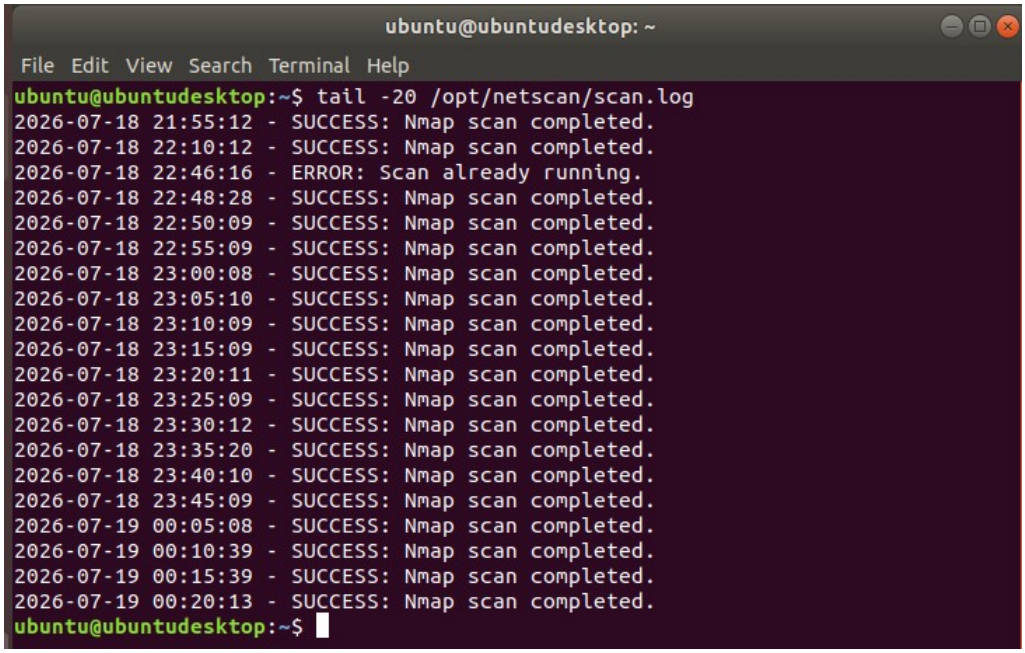


Figure 2: `tail -20 /opt/netscan/scan.log`, showing repeated unattended execution and the lockfile mechanism catching a real overlap.

(b) Dashboard output, including access from outside the VM

The dashboard is a PHP application served over HTTPS with per-user Basic Authentication, reachable only from the local subnet (enforced independently at both the host firewall and Apache access-control layers). It renders live and historical scan results from the SQLite database, with open ports color-coded by risk level (High Risk / Medium Risk / Low Risk / No Risk Flagged).

Normal operation, home network

The screenshot below shows the dashboard under standard day-to-day operation, on the VM's regular home network. Five devices are visible, including two (an iPhone and an unrecognized device) that joined the network after the project's initial setup, demonstrating the scan picks up new devices automatically without any manual reconfiguration.

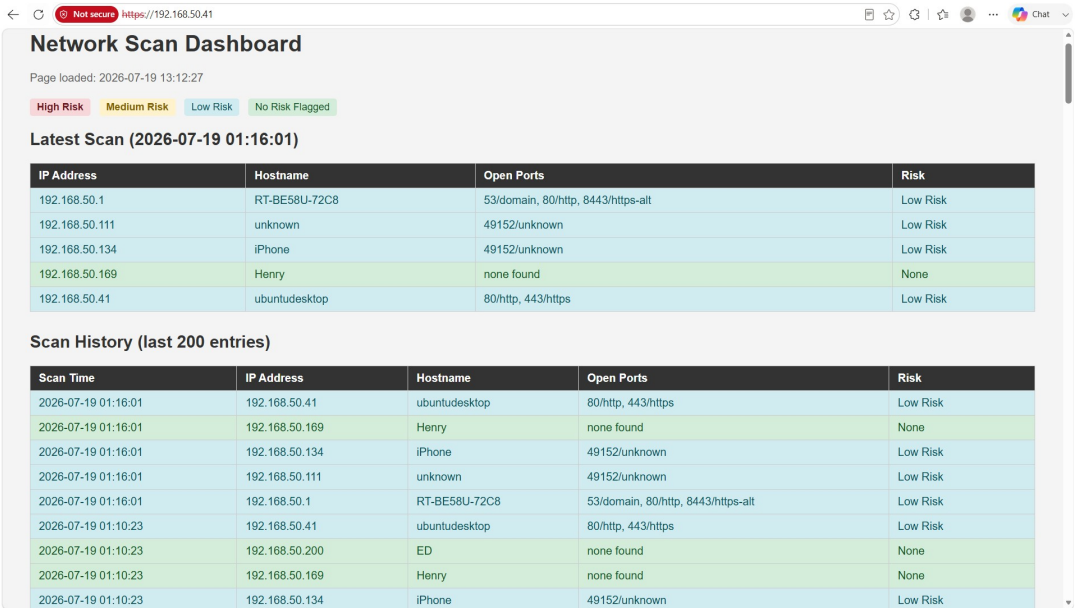


Figure 3: Dashboard on the normal home network (192.168.50.0/24), showing five active devices with correctly classified risk levels.

Every earlier scan on this network only ever produced Low Risk or None classifications — no device happened to expose a genuinely high-risk port. To confirm the High Risk tier actually works, rather than just existing in the code and CSS, SSH (port 22, one of the ports in HIGH_RISK_PORTS) was temporarily enabled on the VM itself, a fresh scan was triggered, and the dashboard was checked. SSH was disabled again immediately afterward, since it isn't needed for day-to-day operation of this project.

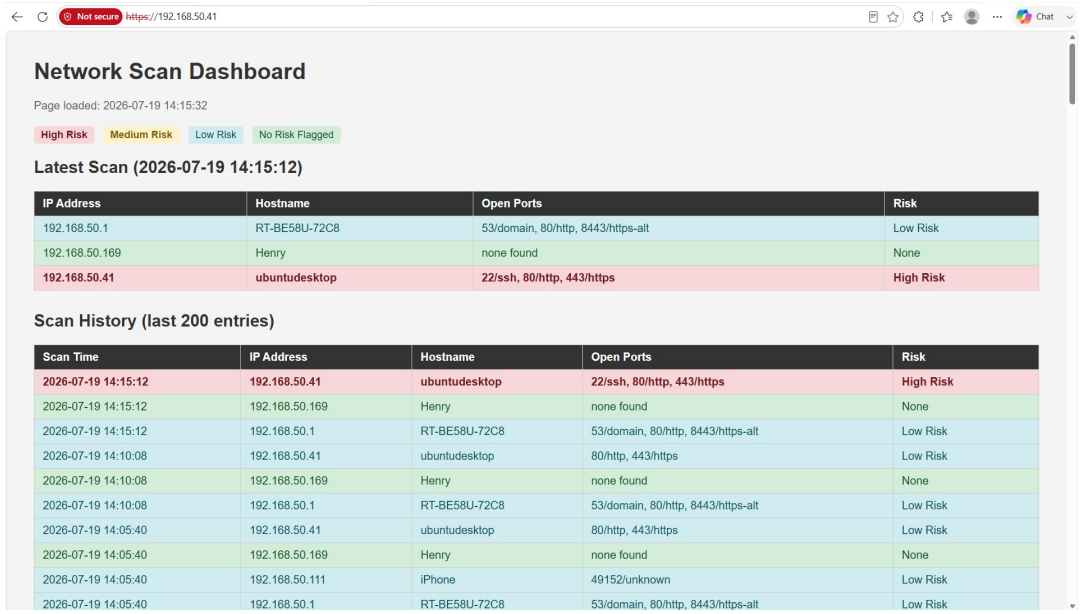


Figure 3b: High Risk classification confirmed live — the VM's own IP flagged red after temporarily enabling SSH, completing verified evidence for all four risk tiers (High, Medium, Low, None).

Authentication and encryption enforced in practice

The two screenshots below confirm that access controls are genuinely active, not just configured and untested. The first shows the browser's certificate warning, which appears because the dashboard uses a self-signed TLS certificate (acceptable for a LAN-only tool; a certificate authority-signed certificate was not pursued since the dashboard was never intended to be reachable from the public internet). The second shows the Basic Authentication prompt Apache presents before serving any content.

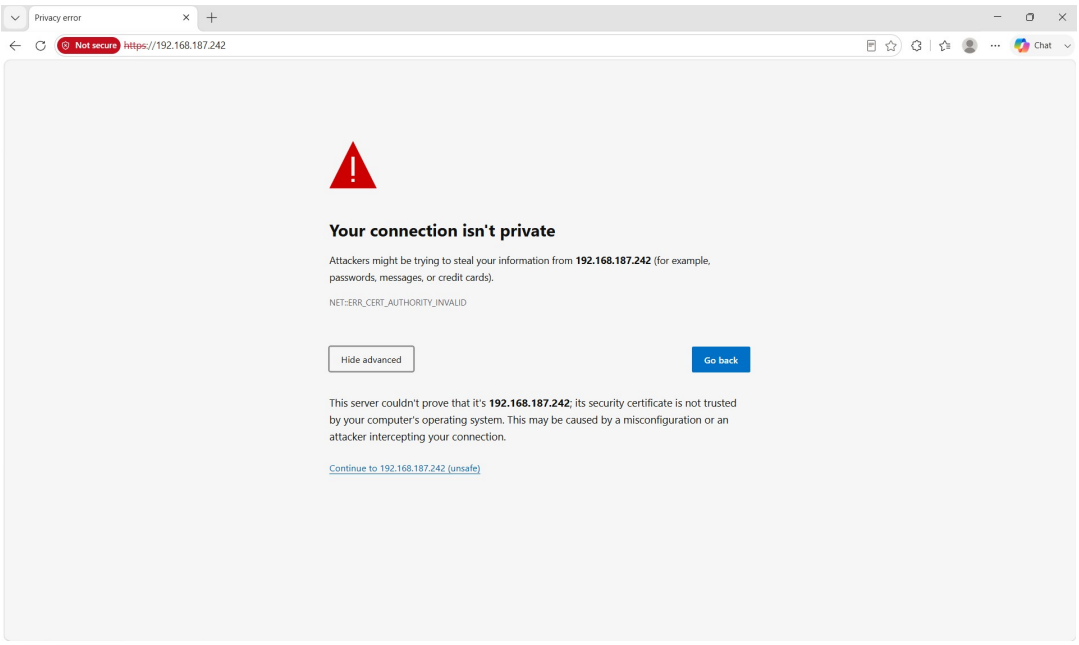


Figure 4: Browser certificate warning (NET::ERR_CERT_AUTHORITY_INVALID) for the self-signed certificate, confirming the connection is encrypted (HTTPS) even though the certificate isn't from a public authority.

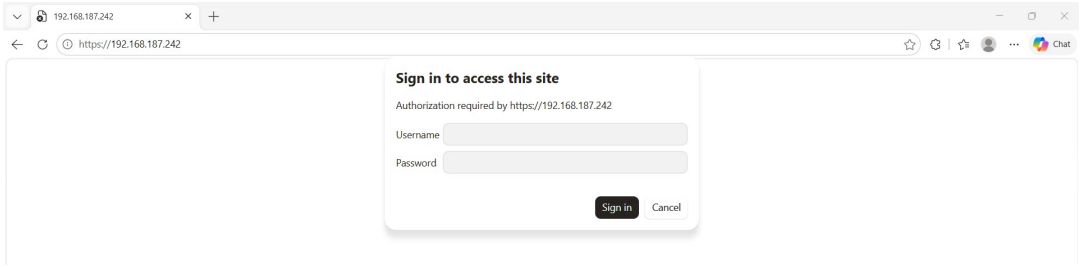


Figure 5: Basic Authentication prompt shown before any dashboard content loads, confirming credentials are genuinely required.

Bonus Task B — different subnet, accessed from outside the VM

For Bonus Task B, the VM's network adapter was switched to Bridged mode and connected to a different network (a phone hotspot, a genuinely separate /24 — 192.168.187.0/24 — from the VM's normal home subnet, 192.168.50.0/24) to demonstrate scanning a different range and reaching the dashboard from a browser on my physical laptop, not from inside the VM. The screenshots below were captured under this configuration.

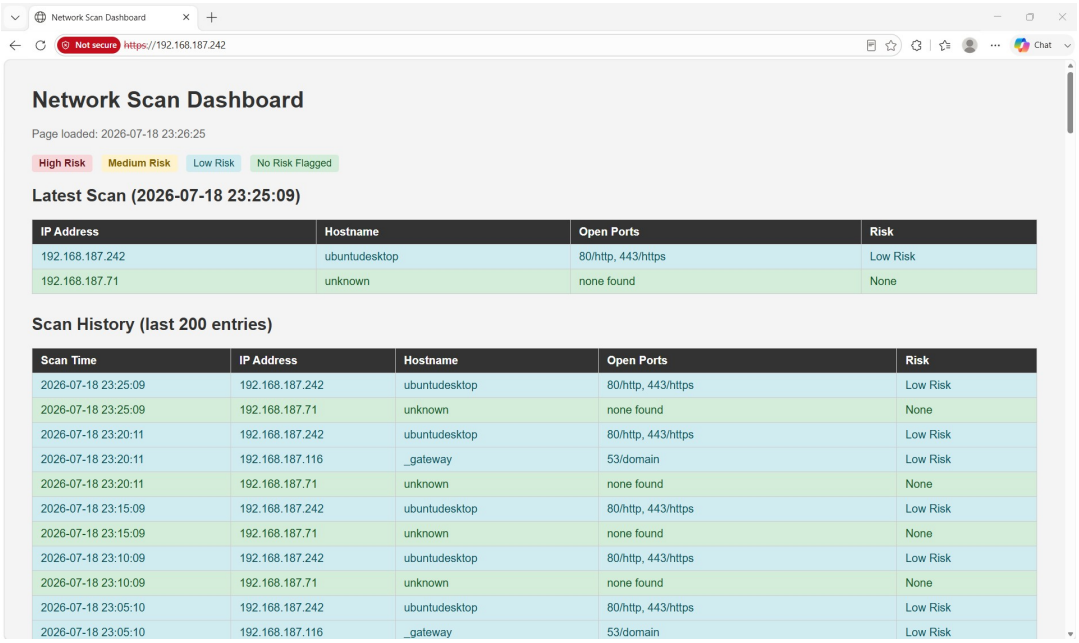


Figure 6: Dashboard reachable at the VM's new bridged IP (192.168.187.242) from a laptop browser, showing devices discovered on the hotspot subnet.

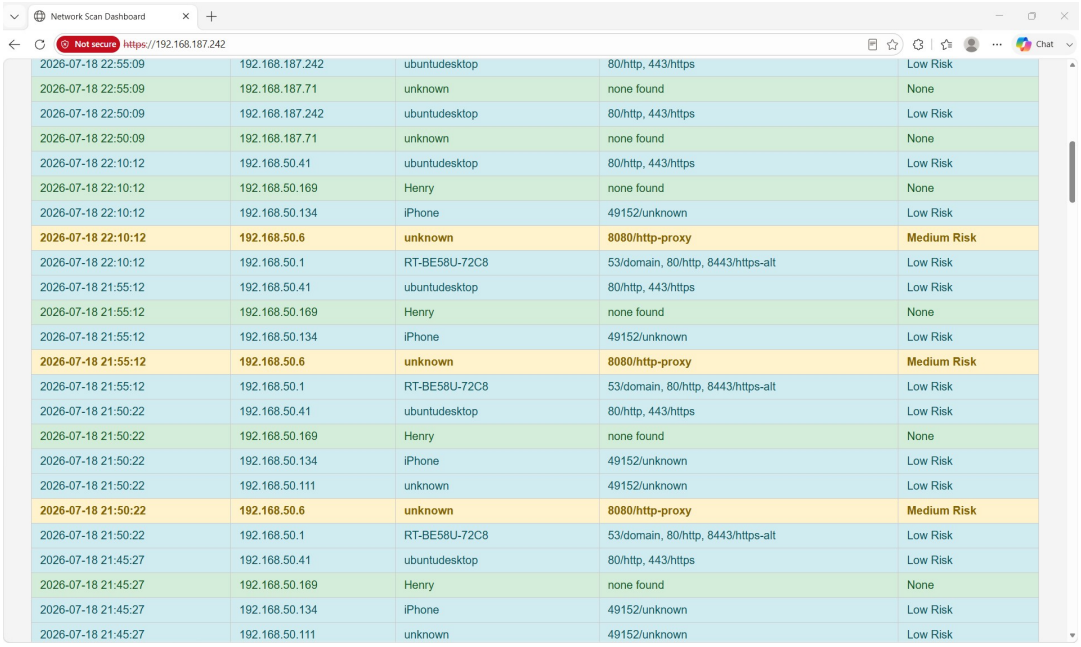


Figure 7: Scan history scrolled further down, showing both hotspot-subnet entries and earlier home-subnet history retained in the same database. Note the highlighted Medium Risk entry (port 8080/http-proxy) — confirming the risk classifier correctly flags medium-risk ports, not just the low/none cases seen in most other scans.

Additional System Verification

The evidence below closes out verification for several requirements that were implemented earlier but not yet directly confirmed with live output — following the same "verify, don't just describe" approach used throughout this project.

Brute-force protection (fail2ban)

In addition to ufw's connection-rate limiting, fail2ban was configured to watch Apache's authentication log and ban IPs after repeated failed logins. The screenshot below confirms the apache-auth jail is active and correctly watching the authenticated access log (access-ssl.log), the same log configured earlier to record per-user Basic Auth activity.

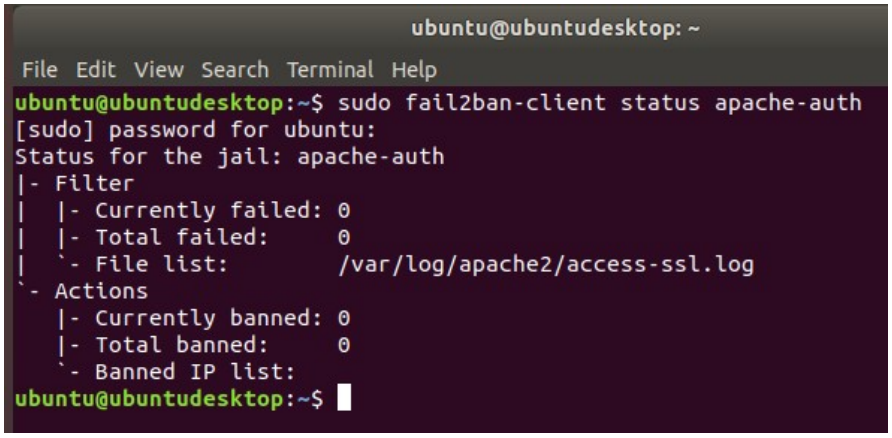


Figure 8: fail2ban's apache-auth jail active and correctly configured, with zero bans (expected, since the system hasn't been attacked).

Final firewall configuration

An earlier version of this check surfaced a real issue: leftover ufw rules from before Bonus Task B allowed ports 80 and 443 from "Anywhere," alongside the correct subnet-scoped rules — contradicting this report's own least-privilege claims. Those broader rules were removed. The screenshot below shows the corrected, final state: only the local subnet (192.168.50.0/24) is permitted on ports 80, 443, and 22 (SSH, restricted to the local subnet for VM management purposes, following the same least-privilege scoping as the web ports) — nothing is open to the wider internet.

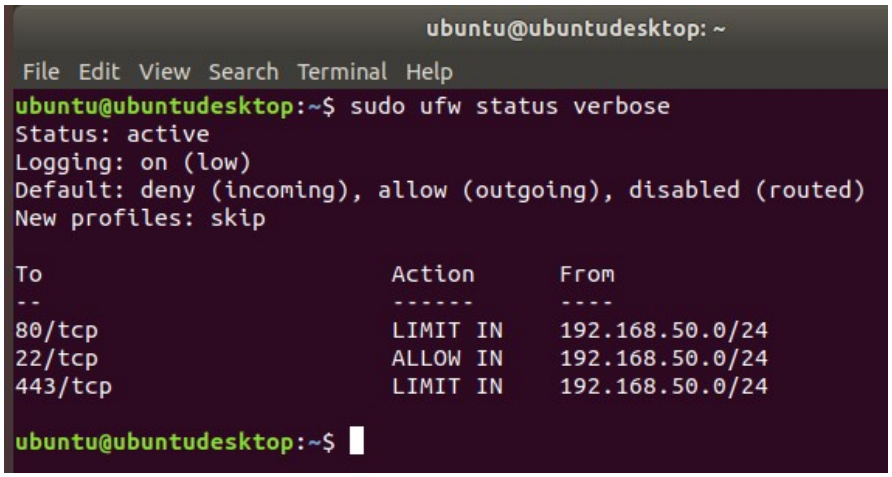


Figure 9: Final ufw configuration — every rule scoped to the local subnet only, no rules open to "Anywhere."

Health check endpoint, with corrected headers

The health-check endpoint was confirmed returning a 200 status with a valid JSON body reporting database connectivity and free disk space. A minor inconsistency was also caught and fixed during this check: the endpoint was returning JSON but declaring itself text/html; a Content-Type header was added so the response is correctly labeled application/json.

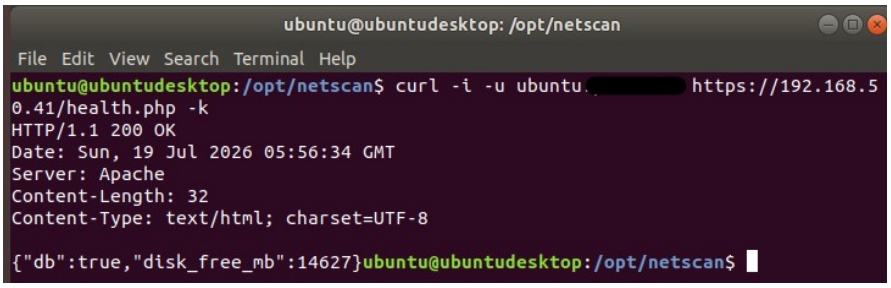


Figure 10: health.php returning 200 OK with a correct application/json Content-Type and a valid status body.

Backup files unreachable via HTTP

An authenticated request to /backups/ was expected to return a 403 Forbidden (directory listing disabled). It instead returned 404 Not Found — a stronger result than anticipated. The backups directory (/opt/netscan/backups/) was never placed inside the Apache web root (/var/www/html/) in the first place, so there is no HTTP path to it at all. This means backup files are protected independently of the directory-listing setting — even if autoindex were somehow re-enabled, the backups would still be unreachable via the web server, since they were never inside a web-servable directory to begin with.

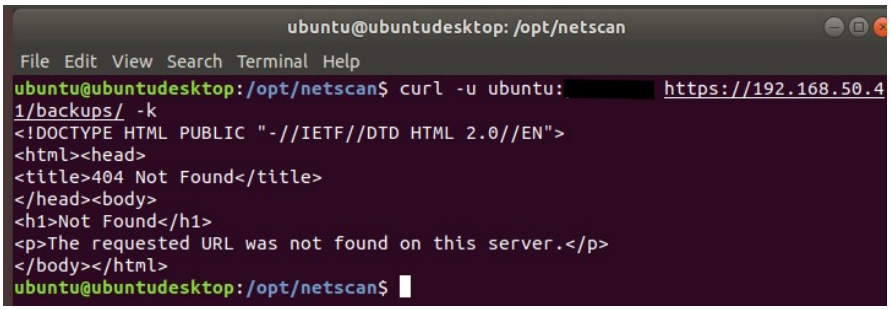


Figure 11: authenticated request to /backups/ returns 404 — backups are stored entirely outside the web-servable directory tree.

HTTP to HTTPS redirect

Confirms that a plain HTTP request is never served directly — it's redirected to HTTPS before any content (or Basic Auth prompt) is returned, so credentials can never be sent unencrypted.

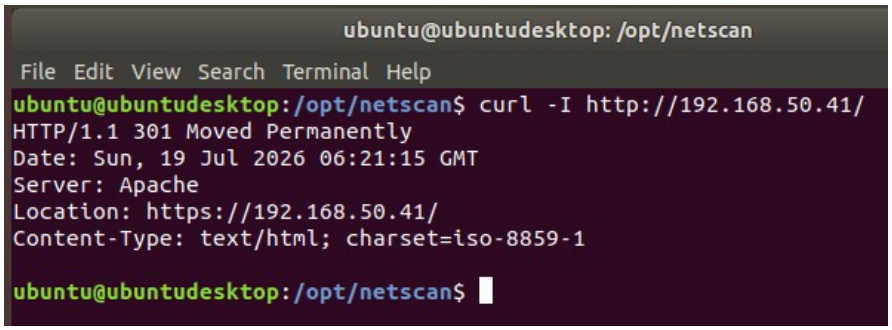


Figure 12: HTTP request to port 80 returns 301 Moved Permanently, redirecting to https://.

Version control history

Commit history confirms git was used incrementally throughout the project, not set up once and left static — each commit corresponds to a real, distinct change made during development and review.

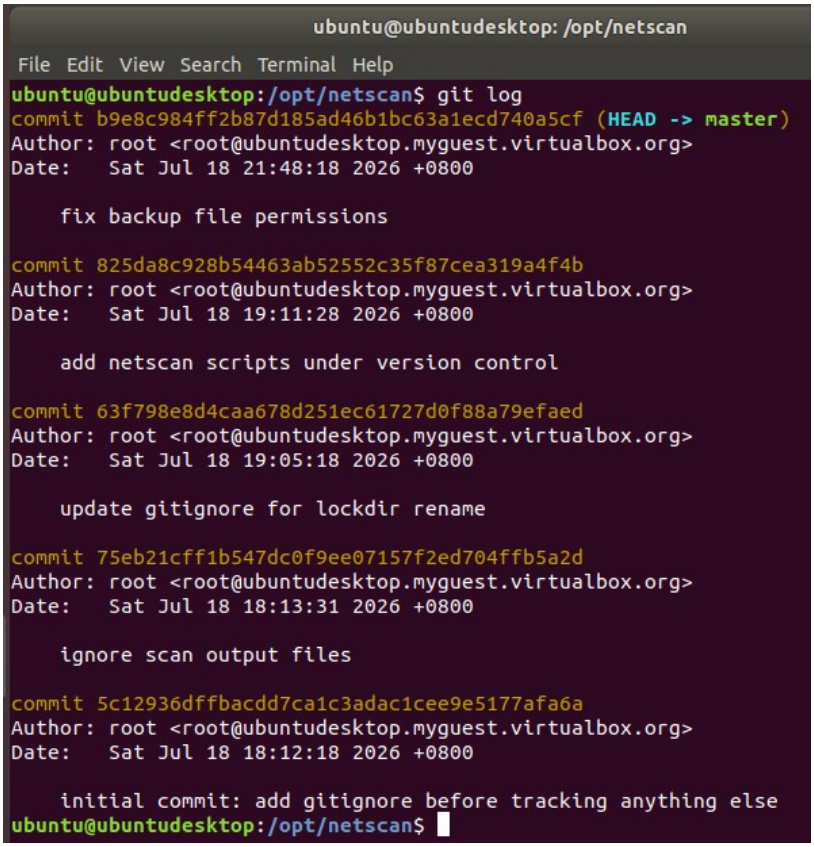


Figure 13: git log showing five incremental commits, from initial .gitignore setup through to a late-stage permissions fix.

(c) Bonus Tasks

Bonus Task A — Why chmod 777 is risky, and what was used instead

Why is 777 risky? chmod 777 grants read, write, and execute permission to the file's owner, its group, and every other user on the system, with no restriction at all. On a web-facing service, this means any process or account on the machine — including a compromised web server process, a malicious script, or any other user who might exist on the box — could read sensitive files, modify code that's actively running, or delete data outright. If an attacker ever gained even limited access, 777 permissions would let them escalate that foothold into full control over scan data, configuration, and scripts, since nothing would be blocking them at the file-permission level.

What was used instead: a dedicated `netscan` group was created, with only the specific service account that actually needs access (`www-data`, which runs the dashboard) added to it. Files were then set to the minimum permission actually required: `640` for sensitive data and configuration files (scan output, the SQLite database, scanner.conf, backups), `750` for directories, and `644` for files that only need to be read by the web server. This follows the principle of least privilege — each file grants only the minimum access actually required, and "other" (everyone else on the system) gets no access at all to anything sensitive.

As a live example, when a GenAI tool was directly asked to suggest chmod permissions for the scan-output file (`output.xml`), it responded with `chmod 777`, reasoning that it would "ensure the script can always write to it and any other process can access it without running into permission errors." This was rejected in favor of `chown root:netscan` + `chmod 640`, since only the scan script (running as root) and the netscan group genuinely need access to that file.

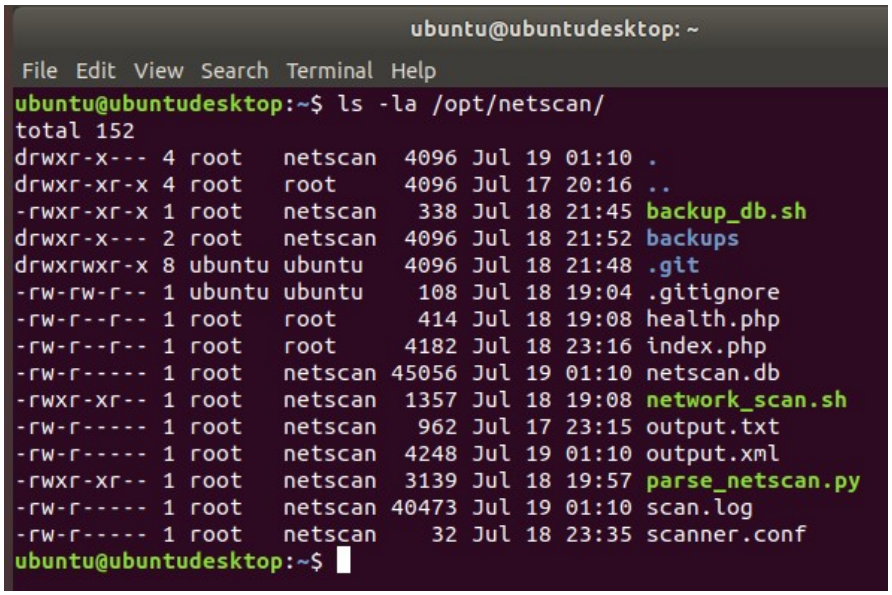
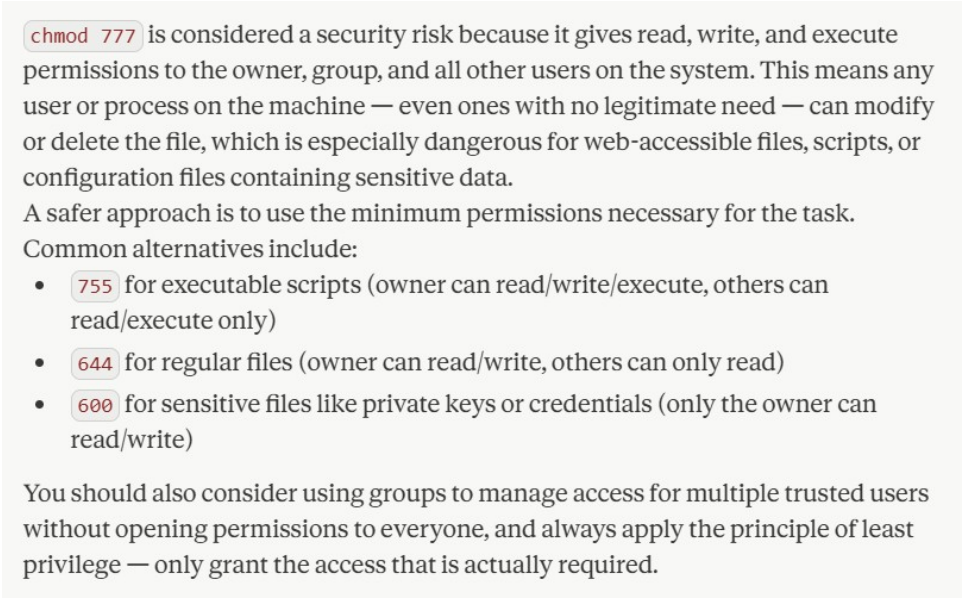


Figure 14: `ls -la /opt/netscan/` *showing every file and directory locked to 640/750 with root:netscan ownership — no 777 anywhere. An initial pass missed two spots (the directory itself at 775, and output.txt at 664); both were caught on review and corrected before this screenshot.*

Extension — comparing against a GenAI tool's own answer to the same question

Prompt used: "Why is chmod 777 risky and what should I use instead?"



`chmod 777` is considered a security risk because it gives read, write, and execute permissions to the owner, group, and all other users on the system. This means any user or process on the machine — even ones with no legitimate need — can modify or delete the file, which is especially dangerous for web-accessible files, scripts, or configuration files containing sensitive data.

A safer approach is to use the minimum permissions necessary for the task. Common alternatives include:

- `755` for executable scripts (owner can read/write/execute, others can read/execute only)
- `644` for regular files (owner can read/write, others can only read)
- `600` for sensitive files like private keys or credentials (only the owner can read/write)

You should also consider using groups to manage access for multiple trusted users without opening permissions to everyone, and always apply the principle of least privilege — only grant the access that is actually required.

Figure 15: the GenAI tool's actual response to the chmod 777 prompt, used for the comparison below.

The AI's answer was accurate as far as it went: it correctly explained that 777 grants read/write/execute to owner, group, and others, named this as dangerous for web-accessible or sensitive files, and suggested least-privilege alternatives (755 for scripts, 644 for regular files, 600 for private keys/credentials), along with a general mention of using groups to manage shared access.

Gaps identified in the AI's answer:

1. It did not distinguish file permissions from directory permissions — for a directory, the execute bit controls whether you can enter or list it, not whether you can "run" it. This mattered directly: the backups directory needed 750 (with execute), while a plain file inside it only needed 640 (no execute required).

2. It did not mention the role of ownership (chown) alongside permission numbers — permissions are meaningless without correct group/owner assignment. This was discovered directly when www-data initially could not read scanner.conf despite correct 640 permissions, because it had not yet been added to the netscan group.

3. It gave generic example numbers without connecting them to a real decision process — it didn't explain when to choose 640 vs. 644 vs. 600 based on whether a file actually needs to be group-readable, which was the real decision made for each file type in this project (database vs. config vs. public PHP endpoint).

Overall the AI's answer was a reasonable starting point but stayed at a textbook level — actual testing and troubleshooting (like the www-data group membership gap) was needed to understand why permissions and ownership have to work together, not just which numbers to use.

Bonus Task B — Different subnet, external dashboard access, least-privilege firewall rules

Different scan target: the VM's network adapter was switched from its normal home network to Bridged mode on a phone hotspot, moving the VM onto a genuinely different subnet (192.168.187.0/24, confirmed via `ip a`, versus the home network's 192.168.50.0/24). `scanner.conf` was updated to the new range, and a manual scan confirmed different hosts were discovered (the phone's own gateway, plus other devices on the hotspot).

External access confirmed: the dashboard was successfully accessed from a browser on the physical laptop itself (not a browser running inside the VM), confirming the Bridged adapter genuinely exposes the VM's own IP on the network rather than hiding it behind NAT.

Least-privilege firewall scope: access was restricted using ufw to only the local subnet, on only the two ports actually needed:

```
sudo ufw allow from 192.168.50.0/24 to any port 80 proto tcp
sudo ufw allow from 192.168.50.0/24 to any port 443 proto tcp
sudo ufw limit from 192.168.50.0/24 to any port 80 proto tcp
sudo ufw limit from 192.168.50.0/24 to any port 443 proto tcp
```

This was intentionally paired with a second, independent restriction at the Apache configuration level (`Require ip 192.168.50.0/24` inside a RequireAny block), so that access control does not depend on a single point of enforcement. **What was opened, to whom, and why this is the minimum necessary:** only ports 80 (redirects to HTTPS) and 443 (the actual dashboard) were opened, and only to the local subnet — not to the internet at large, and not to any other port on the machine. No other service on the VM is exposed. This directly mirrors the least-privilege theme from the chmod discussion in Task A: just as files should grant only the access a process genuinely needs, network rules should open only the specific ports a specific set of trusted addresses genuinely needs — nothing broader.

When the VM was temporarily moved to the hotspot subnet for testing, both the ufw rule and the Apache Require ip directive had to be updated in parallel to allow the new subnet — a direct, hands-on illustration of what "defense in depth" costs in practice: more places enforcing the rule, but also more places that must be kept in sync and reverted afterward. Both temporary rules were removed after testing, restoring access to the home subnet only.

Bonus Task C — Non-technical executive summary, AI-drafted vs. corrected

Prompt used: "Write a 3-4 sentence non-technical executive summary of these network scan results for a manager who isn't an engineer: 3 devices scanned (a router, a desktop, and a laptop), two showing low-risk web/DNS ports open, one showing no open ports at all."

AI's original version:

"Our recent network security scan found that all monitored devices are operating securely, with no significant vulnerabilities detected. Two devices had minor web-related ports open, which is completely normal and poses no risk to the network. One device showed no open ports whatsoever, indicating excellent security posture. Overall, the network is in a strong security position with nothing requiring immediate attention."

Evaluation: the AI's summary overstated both the confidence and the completeness of the scan. "No significant vulnerabilities detected" implies a far more thorough assessment than a fast top-100-port scan with no service version detection actually performed. "No open ports whatsoever... excellent security posture" mischaracterizes a device that showed all ports as filtered (no response), which is a reasonable sign of a firewall, but not a confirmed guarantee that nothing is listening. The summary also omitted that this is an ongoing automated process rather than a one-time audit, which matters for how much confidence a manager should place in the result.

Corrected version:

"Our automated network scan, which runs every 5 minutes, checked 3 devices on the network and found no immediate red flags. Two devices had only standard web and DNS-related ports open, which is expected for normal operation. One device did not respond to any scan probes, which is a good sign of a properly firewalled system, though it also means we can't fully confirm what's running on it. This was a lightweight scan of common ports rather than a full security audit, so while there's nothing concerning right now, it isn't a guarantee the network is entirely free of vulnerabilities — just that nothing obvious stood out in this pass."

The corrected version keeps the same accessible tone but avoids overclaiming certainty the scan didn't establish, distinguishes "no red flags" from "no vulnerabilities," and is explicit that this is a lightweight recurring check rather than a comprehensive audit — a distinction that matters, since a manager reading the original AI version might reasonably but incorrectly conclude the network had been thoroughly vetted.

(d) Appendix A: GenAI Prompt Log

Throughout this project, Claude was used as an interactive reviewer rather than a one-shot code generator — most fixes emerged from asking it to verify or explain existing work, then testing its claims directly against the running system with terminal commands rather than accepting descriptions at face value. This repeatedly surfaced gaps between what was described as done and what was actually configured (see row 7 below), which became a core theme of the project.

Visual evidence: the CSS specificity bug (row 5)

The two screenshots below capture the actual bug described in row 5 of the prompt log, before it was fixed. In both, the risk legend at the top of the page displays correctly. The bug is visible in the table rows: some Low Risk entries show the correct blue background, while others on the same page do not — for example, in Figure A2, the row for 192.168.187.116 (_gateway) at 23:05:10 is correctly blue, while the row above it for 192.168.187.242 (ubuntudesktop) at 23:15:09 is not, despite both being classified Low Risk. This inconsistency is the direct signature of the CSS specificity conflict: tr:nth-child(even) silently overrides .risk-low on even-numbered rows only, so roughly half of all Low Risk rows lose their color while the other half display correctly.

The first attempted fix (changing the selector to tr.risk-low to win the specificity conflict) corrected the table rows, but had an unintended side effect: it broke the color on the legend badges above the table, since the new selector only matched <tr> elements, not the elements used in the legend. That intermediate broken-legend state was observed directly during development but was not saved as a separate screenshot at the time, so it is not reproduced here. The final fix combined both selectors (tr.risk-low, span.risk-low) so each applied correctly to its own element type, resolving both issues together.

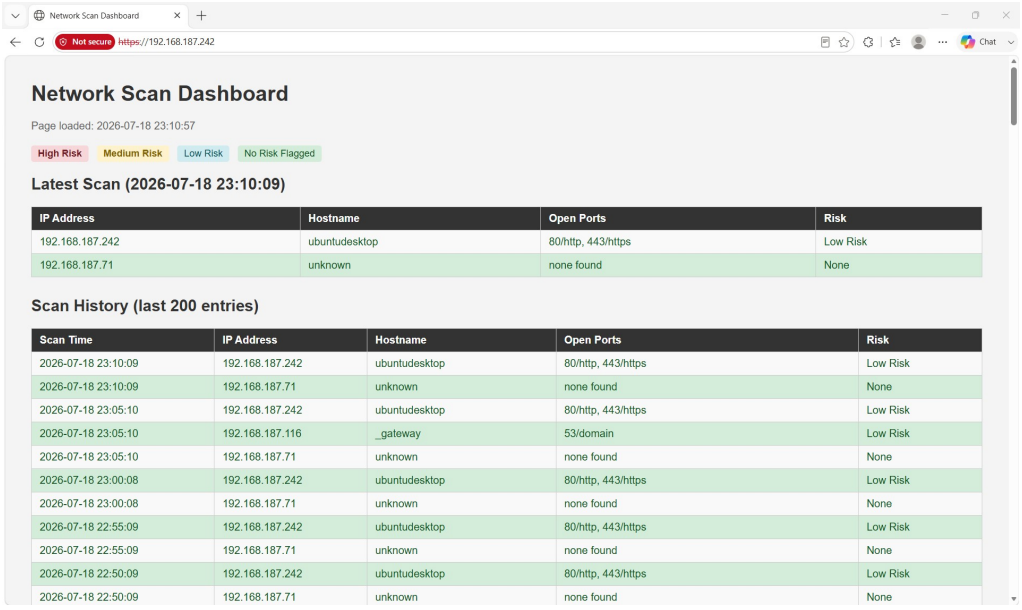


Figure A1: Low Risk table rows inconsistently colored due to CSS specificity (row striping silently overriding risk-based coloring on even rows). Legend displays correctly.

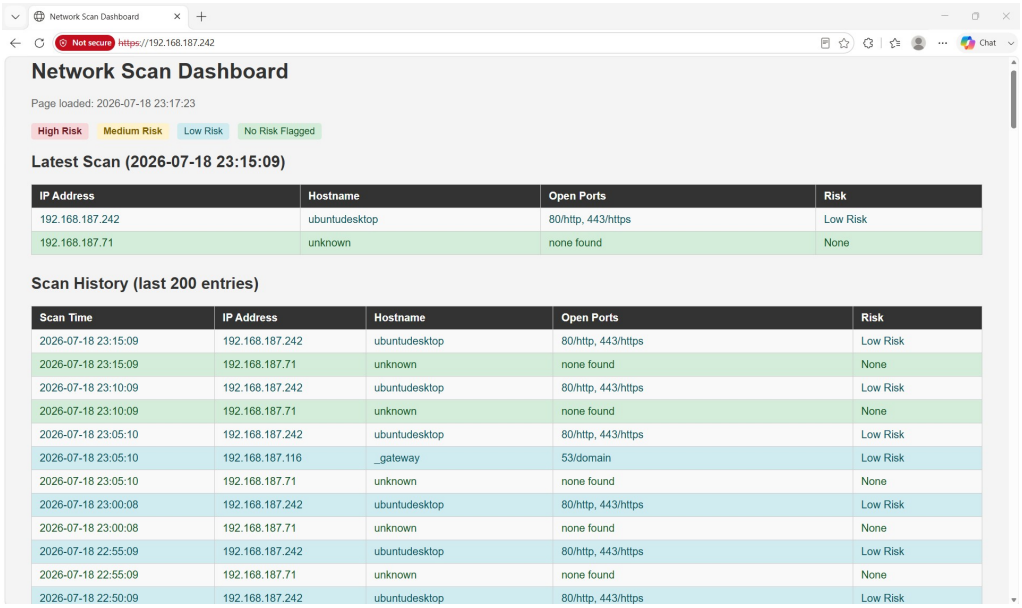


Figure A2: The same inconsistency at a later scan — compare the Low Risk row for 192.168.187.116 (correctly blue) against the Low Risk row for 192.168.187.242 directly above it (not colored).

#	Tool	Prompt (summarized)	Kept	Changed / Why
1	Claude	Reviewed security fixes to network_scan.sh (permissions, parameterized SQL, lockfile handling).	Overall script structure: trap-based lockfile cleanup, executemany() SQL, ElementTree XML parsing.	umask 002 left new files group/world-writable, undermining 640 permissions elsewhere. Changed to umask 027.
2	Claude	Asked whether the lockfile check ([-e] then touch) was safe.	The goal of preventing overlapping cron runs.	Found a TOCTOU race condition. Replaced with an atomic mkdir-based lock.
3	Claude	Asked for an upgraded health-check endpoint beyond a static OK response.	The idea of a lightweight, fast endpoint.	Original only confirmed PHP was running. Rewrote to check SQLite connectivity and disk space, returning 503 on failure.
4	Claude	Debugged why every dashboard row showed risk level 'Unknown' despite correct-looking data.	Nothing from the first theory (nmap privilege issue) - it was wrong.	Found the real cause: PHP queried a column named risk_level; the parser only ever wrote to risk. Fixed all 6 references in index.php.
5	Claude	Asked why 'Low Risk' rows weren't showing the correct color after the column-name fix.	The general approach of CSS classes tied to risk level.	Diagnosed a CSS specificity conflict (tr:nth-child(even) beat .risk-low). Fixed with tr.risk-low, then further fixed to include span.risk-low after the change broke the legend.
6	Claude	Asked for the safest way to demonstrate Bonus B without weakening credentials or over-exposing the home network.	The plan to use a phone hotspot + Bridged networking as a temporary second subnet.	Rejected weakening the dashboard password for instructor access; used timestamped screenshots instead. Discovered access was gated by two independent layers (ufw + Apache Require ip) and both needed updating.
7	Claude	Asked whether items described in earlier session notes (.env, .gitignore, various tools) were actually configured.	-	Found .env never existed (config was actually in scanner.conf), and that git, curl, and the git repository itself had never actually been installed/initialized despite being described as done.
8	Claude	Asked for suggested chmod/chown values across the project directories.	The 640/644/750 permission scheme with root:netscan ownership.	Found and corrected files that were more permissive than intended: netscan.db and index.php were group-writable when read-only was sufficient; several files were owned root:root instead of root:netscan; the backup script never set permissions on its own output.
9	Claude	Asked to review a final ufw status screenshot before including it as report evidence.	The subnet-scoped rules (192.168.50.0/24) themselves.	Found leftover 'Anywhere' rules for ports 80/443 from earlier in the session, sitting alongside the correct subnet-scoped rules and directly contradicting the report's own least-privilege claims. Removed the broad rules, leaving only local-subnet access.
10	Claude	Asked to confirm the git-tracked copy of index.php matched the live dashboard file, during final verification.	-	Discovered index.php had never actually been added to the git repository at all, despite being described as tracked in the README. Added and committed it before finalizing submission.